

MPI 编程实验

——以高斯消去为例

杜忱莹 周辰霏

2021 年 5 月

麦隽韵

2022 年 5 月

李君龙

2023 年 5 月

丁延峰

2024 年 5 月

华志远 孔德嵘 张逸非

2025 年 5 月

王卓然 邹博闻

2026 年 5 月

目录

1 实验介绍	4
1.1 实验选题	4
1.2 实验要求	4
1.3 基础要求	4
1.3.1 高斯消去	4
1.3.2 ANN、NTT 和口令猜测	4
1.3.3 SVD 算子	5
1.4 进阶要求	5
2 多进程与多线程介绍	5
2.1 进程与线程	5
2.2 MPI 多进程	6
3 高斯消去实验设计指导	7
3.1 实验总体思路	7
3.2 MPI 的 C++ 编程	9
3.3 作业注意要点及建议	10
4 ANN 选题指导	11
4.1 IVF (9 分)	11
4.2 图索引 (6 分)	12
5 NTT 选题指导	12
5.1 新的规约优化	12
5.2 常规优化	14
6 口令猜测选题	14
6.1 基础要求	14
6.2 进阶要求	15
7 矩阵 SVD 选题	17
7.1 优化要求	17
7.1.1 实现主从式结构	17
7.1.2 改变现有迭代方法	17

7.1.3	加速和 profiling	18
7.1.4	合并之前实验内容	18
7.2	其他自行探索	18
7.3	特殊情况的说明	18

1 实验介绍

1.1 实验选题

1. 基础选题：高斯消去法 (LU 分解)。
2. 进阶选题：ANN，NTT，口令猜测，SVD 算法四选一。

1.2 实验要求

1.3 基础要求

本次作业的基础要求根据选题不同有所区别，各选题的基础要求及对应得分上限如下。

1.3.1 高斯消去

高斯消去选题的基础要求得分最高为满分的 80%，本次作业中对应最高分为 12 分。该选题要求 ARM 平台（华为服务器）或 x86 平台（自行安装）上进行普通高斯消去计算的 MPI 并行化实验，具体要求如下：

1. 设计实现适合的任务分配算法，分析其性能。
2. 在 ARM 平台或 x86 平台上编程实现、进行实验，测试不同问题规模、不同节点数/线程数下的算法性能（串行和并行对比），讨论一些基本的算法/编程策略对性能的影响。
3. 探讨与多线程（Pthread 或 OpenMP）和 SIMD 的结合。
4. 更多探索：MPI 并行化的不同算法策略（如块划分、循环划分等不同任务划分方法，流水线算法等）及其复杂性分析、体系结构相关优化（如 cache 优化）等。

实验完成后，需要撰写并提交研究报告，要求类似 SIMD 编程作业，篇幅不超过 20 页。

1.3.2 ANN、NTT 和口令猜测

ANN、NTT 和口令猜测三个选题的基础要求得分最高为满分的 90%，本次作业中对应最高分为 13.5 分。其基础要求内容与高斯消去选题类似。

需要注意的是，虽然 ANN、NTT 和口令猜测选题的基础要求分数上限高于高斯消去选题，但这并不意味着实际得分必然更高，最终得分仍将根据具体完成情况、实验质量、性能分析深度和报告撰写质量综合评定。

1.3.3 SVD 算子

SVD 算子选题的基础要求得分最高为满分的 100%，本次作业中对应最高分为 15 分。该选题的基础要求内容与高斯消去选题类似。

需要注意的是，虽然 SVD 算子选题的基础要求分数上限高于其他选题，但这并不意味着实际得分必然更高，最终成绩仍取决于具体实现效果、实验完整性、性能优化程度和研究报告质量。

1.4 进阶要求

参照各选题的对应章节。

2 多进程与多线程介绍

本次实验涉及到的 MPI 编程是多进程编程，什么是进程？本节介绍了线程和进程的区别与联系。首先明确：OpenMP 和 Pthread 均为多线程编程。

2.1 进程与线程

进程 (process) 是操作系统进行资源分配的最小单元，**线程 (thread)** 是操作系统进行运算调度的最小单元。

通常情况下，我们在运行程序时，比如在 Linux 下通过 `./test` 运行一个可执行文件，那么我们就相当于创建了一个进程，操作系统会为此进程分配 ID 和堆栈空间等资源。如果这个可执行程序是个多线程程序（比如由 OpenMP 或 Pthread 编写的 cpp 文件编译而来），那么这个 `./test` 进程在执行到某个特定位置时会创建多个线程继续执行。所以，**进程可以包含多个线程，但每个线程只能属于一个进程。**

如图2.1所示，每个进程有独立的地址空间（外部框表示），同一进程内不同线程（曲线）共享该进程的内存。但是不同进程不共享地址空间，如果一个进程要访问另一个进程的数据就只能通过特定的函数进行通信。

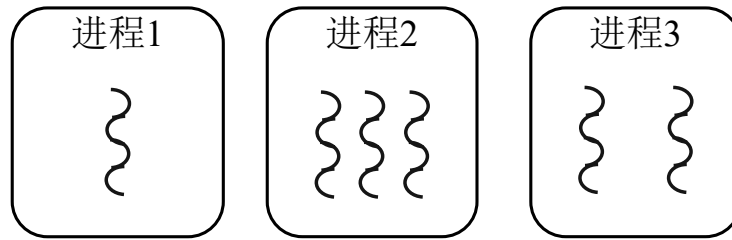


图 2.1: 进程与线程概念

2.2 MPI 多进程

区别于普通的可执行程序只创建一个进程,通过 `mpirun -n num ./test` 运行的程序会根据 `-n` 选项后的参数 `num`, 创建 `num` 个进程运行 `test` 可执行程序。

课程中提到了在开始编写 MPI 部分代码时调用 `MPI_Init` 完成 MPI 初始化,在 MPI 部分结束时调用 `MPI_Finalize` 完成清理工作。这两个函数与多线程中创建和销毁线程有本质的区别: 程序并非只在两个函数调用之间的部分并行执行,而是整体并行执行,在两个函数之间的部分完成数据划分运算、通信等工作。以下方程序为例,代码在两个函数调用的外部定义了 `size` 变量和 `rank` 变量,同时输出 `HelloWorld!`; 在两个函数之间的部分为 `size` 和 `rank` 赋值并输出。编译并通过 `mpirun -n num ./test` 运行后,每个进程均会输出一次 `HelloWorld!`,共输出 `num` 次,这也验证并行并非只发生在 `MPI_Init` 和 `MPI_Finalize` 之间的代码。

```

1  #include "mpi.h"
2  #include <iostream>
3
4  int main(int argc, char* argv[]) {
5      int size, rank;
6      std::cout << "Hello_world!" << std::endl;
7      MPI_Init(&argc, &argv);
8      size = MPI::COMM_WORLD.Get_size();
9      rank = MPI::COMM_WORLD.Get_rank();
10     std::cout << "MPI_size:_" << size << "_rank:_" << rank << std::endl;
11     MPI_Finalize();
12     return 0;
13 }
```

3 高斯消去实验设计指导

3.1 实验总体思路

以高斯消去法为例：

1. 首先初始化生成矩阵元素值，按照之前作业给出的伪代码实现高斯消去法串行算法。

```

1  procedure LU (A)
2  begin
3    for k := 1 to n do
4      for j := k+1 to n do
5        A[k, j] := A[k, j]/A[k, k];
6      endfor;
7      A[k, k] := 1.0;
8      for i := k + 1 to n do
9        for j := k + 1 to n do
10         A[i, j] := A[i, j] - A[i, k] × A[k, j];
11        endfor;
12      A[i, k] := 0;
13    endfor;
14  endfor;
15  end LU

```

2. 使用课堂所学 MPI 编程设计实现合适的任务分配（数据划分）算法，按照不同任务划分方式（如块划分、循环划分等），分别设计并实现 MPI 算法。并考虑将其与 Pthread 算法以及 SIMD 算法结合。对各算法进行复杂度分析（基本的运行时间、加速比的分析以及更深入的伸缩性分析），思考能否继续改进。

以 MPI 按一维（行）块划分消去并行处理为例，假设设置 m 个 MPI 进程、问题规模为 n ，则给每一个进程分配 n/m 行的数据，对于第 i 个进程，其分配的范围为 $[i * (n - n \% m) / m, i * (n - n \% m) / m + (n - n \% m) / m - 1]$ ，而最后一个进程分配 $[(m - 1) * (n - n \% m) / m, n - 1]$ 行。注意，这种简单策略是将余数部分（ $n \% m$ 行）都分配给了最后一个进程，大家可思考稍复杂些但负载更为均衡的分配策略——将余数部分均匀分配给前 $n \% m$ 个进程，每个进程一行。初始化矩阵等工作由 0 号进程实现，然后将分配的各行发送给各进程。在第 k 轮消去步骤，负责第 k 行的进程进行除法运算，将除法结果一对多广播给编号更大的进程，然后这些进程进行消去运算。消除过程完成后，可将结果传回 0 号进程进行回代，也可由所有节点进行并行回代。

一维块循环划分略微复杂，主要是以更小任务粒度循环给各进程分配

Algorithm 1: MPI 版本的普通高斯消元

Data: 系数矩阵 $A[n,n]$, 进程号 $myid$, 负责的起始行 $r1$, 负责的终止行 $r2$, 进程总数 num

Result: 上三角矩阵 $A[n,n]$

```

1 for  $k = 0$  to  $n-1$  do
2   if  $r1 \leq k \leq r2$  then
3     for  $j = k + 1; j < n; j++$  do
4        $A[k,j] = A[k,j] / A[k,k];$ 
5      $A[k,k] \leftarrow 1.0;$ 
6     for  $j = 0; j < num; j++$  do
7        $MPI\_Send(&A[k,0], n, MPI\_FLOAT, j, ...);$ 
8   else
9      $MPI\_Recv(&A[k,0], n, MPI\_FLOAT, j, ...);$ 
10  for  $i \leftarrow r1$  to  $r2$  do
11    for  $j = k + 1; j < n; j++$  do
12       $A[i,j] \leftarrow A[i,j] - A[k,j] * A[i,k];$ 
13     $A[i,k] \leftarrow 0;$ 
14   $k++;$ 
15  ...

```

任务行号的计算会更复杂些，大家自行推导。一维列（循环）块划分与一维行划分略有不同，在除法阶段，需要持有对角线上元素的进程将其广播给其他进程，然后所有进程对自己所负责的列元素进行除法计算；接下来无需广播除法结果，因为需要除法结果的后继行都由同一个进程负责，直接在本地进行消去计算即可。这里就有一些可优化之处，大家可以考虑采用非阻塞通信、单边通信等手段降低对角线元素广播带来的进程空闲等待时间。

二维划分就更为复杂一些，大家需要更小心地计算每个进程负责的行号、列号，安排好通信，显然，采取二维划分后，既需要列方向广播对角线元素，也需要行方向广播除法结果。

流水线算法和普通的块划分的区别在于一个进程负责行的除法运算完成之后，并不是将除法结果一对多广播给所有后续进程，而是（点对点）转

发给下一个进程；当一个进程接收到前一个进程转发过来的除法结果时，首先将其继续转发给下一个进程，然后再对自己所负责的行进行消去操作；当一个进程对第 k 行完成了第 $k - 1$ 个消去步骤的消去运算之后，它即可对第 k 行进行第 k 个消去步骤的除法操作，然后将除法结果进行转发，如此重复下去，直至第 $n - 1$ 个消去步骤完成。

3. 实验方面，改变矩阵的大小、进程数等参数，观测各算法运行时间的变化，对结果进行性能分析。测试相同并发度（总线程数）下不同节点数与每节点线程数的组合，并借助 Vtune profiling 等工具分析算法过程中的同步开销和空闲等待等。

3.2 MPI 的 C++ 编程

1. 头文件

所有进行 MPI 调用的程序单元必须包括“mpi.h”头文件。该文件定义了一些 MPI 常量，并提供了 MPI 函数原型。

```
1 #include "mpi.h"
```

2. MPI 预定义数据类型

```
1 MPI_CHAR
2 MPI_SHORT
3 MPI_INT
4 MPI_LONG
5 MPI_UNSIGNED_CHAR
6 MPI_FLOAT
7 MPI_DOUBLE
8 ...
```

3. 常用的 MPI 函数

```
1 MPI_Comm_size//报告进程数
2 int MPI_Comm_size(MPI_Comm comm, int *size);
3
4 MPI_Comm_rank//报告识别调用进程的 rank，值从 0 size-1
5 int MPI_Comm_rank(MPI_Comm comm, int *rank);
6
7 MPI_Init//令 MPI 进行必要的初始化工作
8 int MPI_Init(
9     int* argc_p /* 输入/输出参数 */,
10    char *** argv_p /* 输入/输出参数 */);
```

```
11
12 MPI_Finalize//告诉 MPI 程序已结束，进行清理工作
13 int MPI_Finalize(void);
14
15 MPI_Send//基本（阻塞）发，向一个进程发送数据
16 int MPI_Send(
17     void* buf /* 存储数据的缓冲区地址 */，
18     int count /* 发送的数据量 */，
19     MPI_Datatype datatype /* 数据类型 */，
20     int dest /* 目的进程编号 */，
21     int tag /* 标识向同一个目的进程发送的不同数据 */，
22     MPI_Comm /* MPI集群的通信域标识 */);
23
24 MPI_Recv//基本（阻塞）接收，从一个进程接收数据
25 int MPI_Recv(
26     void* buf /* 存储数据的缓冲区地址 */，
27     int count /* 接收的数据量 */，
28     MPI_Datatype datatype /* 数据类型 */，
29     int source /* 源头进程编号 */，
30     int tag /* 标识从同一个源头进程接收的不同数据 */，
31     MPI_Comm /* MPI集群的通信域标识 */，
32     MPI_Status *status /* 可以记录更多额外的信息 */);
```

更多更详细的内容请参考 MPI 的讲义。

3.3 作业注意要点及建议

1. 矩阵数值初始化问题

根据有些同学们反映，自己初始化矩阵在计算过程中会出现 inf 或 nan 的问题。这是由于精度问题以及非满秩矩阵造成的。inf 或 nan 的情况无疑会影响结果正确性的判断，也会在并行计算性能上造成一定影响，而随机生成数据的方式很明显无法保证能避免该问题尤其在规模巨大的情况下。个人建议初始化矩阵时可以首先初始化一个上三角矩阵，然后随机的抽取若干行去将它们相加减然后执行若干次，由于这些都是内部的线性组合，这样的初始数据可以保证进行高斯消去时矩阵不会有 inf 和 nan。

2. 不同任务划分策略

前面介绍了 MPI 任务分配（数据划分）策略，在此基础上，可结合多线程以及 SIMD 并行进行任务划分。例如，对于一维行划分，可看作是第二层循环拆分，分配给不同进程；这样，继续进行多线程并行，即可继续对第二层循环进行划分，即，将进程负责的行划分给内部的多个线程，也可

以对最内层循环进行划分，即，将进程负责的所有行的不同列分配给不同线程；而再继续结合 SIMD 并行化，则只能对最内层循环进行向量化。MPI 列划分、二维划分下与多线程和 SIMD 的结合类似，大家自己思考。

3. 计算误差与程序正确性

有关问题规模和并行计算由于重排了指令执行顺序和计算机浮点数所导致误差问题说明参考之前实验。

由于 n 较小的时候测出的计算时间也较小，误差较大，所以建议采取多次测量取均值的方法确定较合理的性能测试结果，同时保证几种算法重复次数一致，减少误差。而且考虑到实验平台等因素，计时测试工具可以考虑使用 MPI 计时。

4 ANN 选题指导

ANN 的 MPI 实验的重点在于探索在多进程上的数据划分和并行搜索，因此在实现时需要重点关注：MPI 进程划分方式、多进程之间的负载均衡、merge/reduce 开销、MPI 与 SIMD/多线程混合并行、通信与计算重叠、分布式 cache locality 问题以及自始至终需要关注的 recall-latency trade-off 问题，可以在实验中多思考这些问题并展现在报告里面，下面是一些基础的 MPI 并行方法，也可以自行探索其他合理的 MPI 并行 ANN 方案。

4.1 IVF (9 分)

对于 IVF 算法，MPI 实现思路和多线程基本一致，大家可以直接在之前的 IVF-SIMD、多线程实现基础上迁移到 MPI 版本。

最基础的方法是将 base data 划分到不同 MPI 进程，每个进程维护自己的一部分 inverted list，当查询到来时被广播到所有 MPI 进程，每个进程本地完成搜索并维护局部 top-p，最后由一个进程进行 top-k 的 merge/reduce (可以选择之前实现 IVF-SIMD 或者 IVF-PQ-SIMD 进行迁移实现，3 分)。

在实现完上面基础 MPI 并行搜索与 top-k merge 之后，可以进一步结合 pthread 或者 openmp 进行混合并行，例如可以进程间 MPI 并行，进程内 query 级并行、簇级并行、rerank 阶段并行等等，在实现时需要尤其注意进程数、线程数的设置，不一定越多越好 (需要考虑启动和通信开销)，同时需要注意进程/线程的负载均衡，并在报告里体现这些思考。(MPI +

OpenMP/Pthread 混合同行 4 分, 负载均衡、merge/reduce、通信优化等进一步优化分析 2 分, 共 6 分)

4.2 图索引 (6 分)

对于图索引算法, 可以尝试使用 IVF+HNSW, 即先对数据集构建 IVF 索引, 在 IVF 每个簇中建立 HNSW。在搜索时簇之间使用 MPI 并行搜索, 如果实现了 HNSW 的查询内并行搜索, 可以在簇内再利用多线程并行加速图索引的搜索。(2 分)

另一个可以探索的算法是直接将数据集划分为 P (P 可以设置为 2-8, 划分方式可以选择随机划分或者使用一些启发式的策略) 个部分, 然后对每个部分单独建立 HNSW, 搜索时利用 MPI 在 HNSW 间进行并行搜索, 最后进行 top-k 的 merge。(2 分)

除了上述介绍的内容, 同学们可以自由讨论其他 IVF 和 HNSW 的组合策略, 比如指导书里面写的 HNSW-on-HNSW, 基本思想都是对数据集做进行一些划分, 然后在不同的划分间并行搜索。(2 分)

上述部分并行方法如果出现负优化现象, 同学们只需在报告中如实汇报实验结果, 并给出合理分析即可。

5 NTT 选题指导

5.1 新的规约优化

目前多数同态加密库里的模数优化并非 Montgomery 规约, 因为其在非 SIMD 加速条件下加速比没有 Barrett 规约高, 适用性也普遍不如。

Barrett 模乘是一种近似算法, 它的主要思路是使用乘法和位移代替取模运算中的除法。

取模显然有下列式子:

$$x \bmod q = x - \lfloor xs \rfloor q$$

其中 $s = \frac{1}{q}$, 如果能以高精度求出 s , 则此公式成立。

Barrett 模乘取 $r = \left\lfloor \frac{2^k}{q} \right\rfloor$, 则使用近似 $\frac{r}{2^k} \approx \frac{1}{q}$, 可得

$$x \bmod q = x - \left\lfloor \frac{xr}{2^k} \right\rfloor q$$

由于近似过程中 $\frac{r}{2^k}$ 与 $\frac{1}{q}$ 始终存在误差, 由 $r = \left\lfloor \frac{2^k}{q} \right\rfloor$ 可得:

$$r = \frac{2^k}{q} - e \Rightarrow \frac{r}{2^k} = \frac{1}{q} - \frac{e}{2^k} \text{ for some } e \in [0, 1)$$

推出 $\frac{r}{2^k} \leq \frac{1}{q}$

默认 k 较大, 一定满足 $x \leq 2^k$, 则对于 $\lfloor \frac{x \cdot r}{2^k} \rfloor$, 一定满足:

$$\lfloor \frac{x \cdot r}{2^k} \rfloor = \lfloor \frac{x}{q} - \frac{x e}{2^k} \rfloor \in \{ \lfloor \frac{x}{q} \rfloor, \lfloor \frac{x}{q} \rfloor - 1 \}$$

因此当前最终输出为

$$x \bmod q = x - \lfloor \frac{x r}{2^k} \rfloor q \in [x \bmod q, (x \bmod q) + q]$$

也就是说, 结果最多偏大一个 q . 所以最终只需要:

```
1 if (res >= q) res -= q;
```

当 $x = a \times b$ 时, 输出又可化为:

$$ab \bmod q = ab - a \lfloor \frac{br}{2^k} \rfloor q \in [b \bmod q, (b \bmod q) + q]$$

用 Barrett 代码计算 $a * b \% q$ 的过程如下:

```
1 uint64_t z = (uint64_t)a * b;
2 uint64_t t = (__uint128_t)z * r >> k; // x*r/(2^k)
3 uint64_t res = z - t * q;
4 if (res >= q) res -= q;
```

在应用 Barrett 模乘时一般取 $k = 64, 2^k = 2^{64}$, 以提高近似精度和模数 q 的最大表示, 由于 unsigned long long 的最大值为 $2^{64} - 1$, 所以需要转化为 __uint128, 转化的过程会占据大量的时间开销, 同时受此影响, 如果模数是大模数, 需要用到高精度类或者其他高精度处理方法 (因为只有这里会超过 ull)。

由上可知, 在不使用 SIMD 的情况下, Barrett 包含更少的状态转换, 更加快捷。而 Montgomery 规约需要将数据转化到 Montgomery 域, 增加了 reduce 次数, 反而较慢。但在 SIMD 和 GPU 优化场景下, Montgomery 规约更加规则, 易于加速。

5.2 常规优化

很容易发现 mpi 优化与多线程的 pthread 优化十分相近, 因此只需要将你在 pthread 里实现的 dif/dit 和 crt 合并在多进程里再实现一次即可, 当然可以尝试将 crt 合并的模数分组计算, 同一进程内使用多个线程进行计算, 最后多个进程再统一合并。

当然需要注意如果实现了四分 NTT, 需要对于奇数进行额外一次的变换才能继续四分。

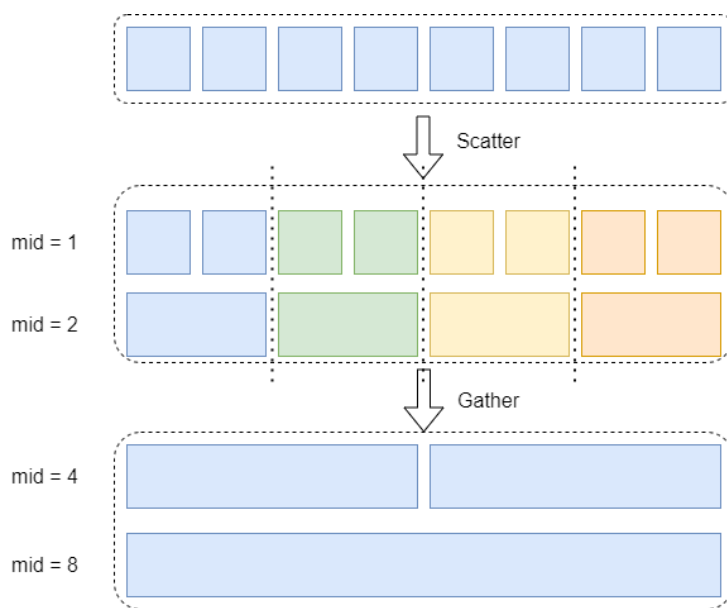


图 5.2: 多进程 NTT 的一种简要实现

6 口令猜测选题

MPI 编程在形式上和 pthread 没有太大区别, 本次实验的重点是了解 MPI 编程的过程和细节, 并且在工程上尝试对口令猜测算法进行并行化。

6.1 基础要求

按照先前多线程的基础要求, 实现口令猜测算法的并行化。并满足以下要求:

1. 你要确保你的并行是多个进程分工完成一个任务，而不是将一个任务复制成多份，让多个线程重复完成。这种错误实现不仅无法带来加速还会造成计算资源的浪费。
2. 对于训练部分，如果你没有完成他的正确并行，至少不要出现如第一点所说的错误实现，具体表现为多个进程在控制台交替输出他的训练进度。
3. 在报告中对你能/不能加速的原因进行分析。

6.2 进阶要求

1. 尝试使用多进程编程，在 PT 层面实现并行计算。先前的并行算法是对于单个 PT 而言，使用多进程/多线程进行并行的口令生成，**现在请尝试一次性从优先队列中取出多个 PT，并同时进行口令生成。**

提示：需要注意每个 PT 生成之后均需要将产生的新 PT 放回优先队列，如果一次性取出多个 PT，那么等待各 PT 生成完成后，再将一系列新的 PT 挨个放回优先队列。

2. 尝试使用流水线的方式补偿通讯开销，将口令的生成任务和哈希任务分配给不同的进程。

生成1		生成2		生成3		生成4		
		加密1-1		加密1-2		加密1-3		
		加密2-1		加密2-2		加密2-3		
		加密3-1		加密3-2		加密3-3		

图 6.3: 流水线示例

如图6.3所示，先前的口令猜测/哈希过程是串行的，也就是猜测完一批口令之后，对这些口令进行哈希，哈希结束之后再继续进行猜测，周

而复始。如果采用多进程（多线程理论上也可以）编程，就可以在猜测完一批口令之后，对这批口令进行哈希，但同时继续进行新口令的生成。第一轮口令哈希结束、第二轮猜测结束之后，再同时进行第二轮口令哈希、第三轮口令猜测。这里的生成和加密使用的进程数并不固定，你可以通过实验探索一下哪个更快。（示例中的 1:3 仅供参考）。

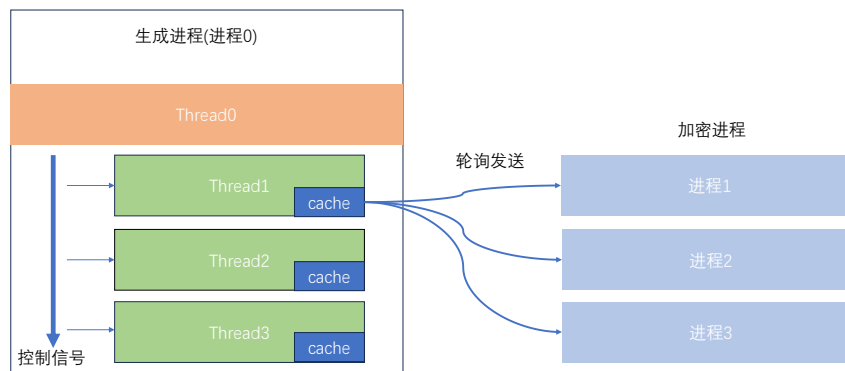


图 6.4: 生成-加密并行流水线架构图

同时你可以考虑在多进程的基础上在每个进程内使用多线程。如图6.4所示，每个进程开四个线程，其中生成端分出一个线程进行总控工作，三个线程进行生成工作，加密端共 3×4 个线程进行加密工作。这个线程分配同样不固定，它取决于你具体的生成和加密速度，整个过程类似生产者-消费者模型，生成进程负责制造口令，而加密端负责消费，平衡好生成与加密的速度才能获得更好的性能。

注意：临近期末，不要在这个进阶要求上死磕，见好就收。即便没有完全实现，也可以将实现的过程和遇到的困难记录在实验报告里。

3. 探索实现其它可以有效减少通讯开销的方法。

上述进阶要求供大家参考。如有其它额外工作，将根据难度、工作量进行给分。

7 矩阵 SVD 选题

矩阵 SVD 分解的多进程实验内容与多线程保持一致，仍旧是针对子矩阵划分的递归过程进行优化，具体可参见多线程实验手册。总的来说，MPI 实验的目的是让同学们学习熟悉相关 API 的调用。

与多线程实验不同的是，多进程实验存在以下难点：

- (1) 中各进程的内存空间是独立的，这导致数据拷贝开销极大；
 - (2) 且进程间的通信并不“轻松”（实际上你可以写一个 pingpong 程序，自己感受一下通信的用时）；
 - (3) 进程结果合并时的竞争问题更加严重。
- 这些问题导致 MPI 实验的加速可能非常困难。

7.1 优化要求

7.1.1 实现主从式结构

对这个矩阵分发和回收过程，一个自然的想法是采用主进程和多个从进程的结构。主进程负责分发及回收，从进程内部进行计算。与多线程类似，多进程依旧存在负载严重不均的问题。因此本次实验依旧需要你实现一个任务池。分配一个进程管理池子，其他进程空闲时来领取任务。

请注意，这种管理的性能或许不如一些更简单的方法，但在尝试其他方法前，至少需要完成任务池。

7.1.2 改变现有迭代方法

在上次实验的手册中，我们的每轮循环过程是：扫描并分割子矩阵 → 得到子矩阵 vec → 从 vec 中逐个取子矩阵，每个矩阵计算一轮 bulge chase 过程 → 回到扫描步骤。

不难发现，扫描过程每轮都会把 1x1 的矩阵再扫一遍，而它们理论上已经不需再计算。所以本实验需要你更改迭代的逻辑：从子矩阵池中取一个非 1x1 的子矩阵 → 对这个子矩阵反复进行 bulge chase 直至可以再次分割 → 再次分割后，将得到的矩阵塞回矩阵池。

助教认为这个迭代过程在数学上是没有问题的，且它的实现难度并没有此前想象中高，所以请你实现。如果你认为新的迭代方法在数学上不正确，欢迎联系助教讨论，**不会因你的证明是错的面扣分**。

7.1.3 加速和 profiling

从往年情况来看，本次实验同学们有很大概率无法加速，这不影响分数，但需要同学们对子进程内存的申请、进程通信过程、进程数据拷贝的开销进行分析。本实验不强制要求使用 perf 等程序，你可以使用计时等方法进行较浅层的考察。

但与之之前相同，无数据的纸上谈兵不会被认为完成了 profiling 任务。如果实现加速，请在报告中注明，会有明确加分，当然本实验助教会非常仔细地检查运行数据。

7.1.4 合并之前实验内容

请继续将 SIMD 实验的代码合并到本次实验中。多线程实验如果你只优化了迭代函数，可以不合并（两次实验的目标函数是一样的），请确保合并后代码的正确性。

此外，请探索一下：只运行 SIMD 和 MPI+SIMD 的性能有差异吗？会不会导致 SIMD 的效果变差？尝试思考原因。

7.2 其他自行探索

如果你自己做了一些额外优化（比如针对拷贝过程），请一并写在报告中。有很多同学询问 xx 任务是否能写在报告里，这里统一回复一下：只有写在报告里，助教才能了解你的劳动成果。

7.3 特殊情况的说明

在多线程实验中，部分同学反映，迭代中分割出的子矩阵规模差异极大，甚至出现只有一个非 1x1 的大矩阵，其余矩阵均 1x1 的情况。这种情况是选题本身性质导致的，发生的概率不能忽略。

不幸的是，即使按本实验手册修改迭代的方式，依旧无法避免其对性能的影响。毕竟还是会回到只有一个进程干活，其他人干瞪眼的情况。

如果你在代码中发现这类问题，允许在其他地方进行多进程优化。但请注意两点：（1）其他函数片段的优化不能与 simd/多线程实验重合，否则最后实验合并时对该函数你要选择哪个版本呢？（2）需要在报告里以数据论证：对很多种子，矩阵都出现这种极不均衡的情况。请给出统计截图，以及你测试的种子号，以供助教检查。

否则助教认为是本次实验重点部分未完成，会扣除大量分数。